Στη γραμμή 4 διαβάζουμε μια γραμμή δεδομένων. Θα μπορούσαμε να χρησιμοποιήσουμε τη συνάρτηση next(), μιας και το όνομα είναι τύπου String. Ωστόσο, αν ο χρήστης καταχωρήσει όνομα και επώνυμο διαχωρισμένα με διάστημα, η next() θα διαβάσει μόνο μέχρι το διάστημα, δηλαδή μόνο το όνομα. Ακόμη χειρότερα, η επόμενη είσοδος που επιχειρείται με την nextInt στη γραμμή 7 θα βρει ως δεδομένα εισόδου το επώνυμο του χρήστη. Θα βρει δηλαδή ένα String, ενώ περιμένει int, οπότε θα παραχθεί λάθος χρόνου εκτέλεσης.

Εφόσον διαβάσουμε σωστά το ονοματεπώνυμο του χρήστη, προχωράμε στην είσοδο του αριθμού μητρώου. Αν εδώ ο χρήστης δεν δώσει ακέραια τιμή, επίσης θα παραχθεί λάθος χρόνου εκτέλεσης. Στη συνέχεια, στη γραμμή 10, διαβάζουμε έναν αριθμό τύπου double.

Το παράδειγμα κλείνει με την in.close() σχολιασμένη. Προσοχή, μην κλείσετε ποτέ ένα αντικείμενο Scanner που είναι σχετισμένο με την System.in. Αν το κάνετε, η εφαρμογή σας δεν θα είναι σε θέση να ανοίξει ροή επικοινωνίας με το πληκτρολόγιο παρά μόνο αν επανεκκινηθεί [3].

Βέβαια, η σωστή χρήση της Scanner απαιτεί τη διαχείριση των λαθών που ενδέχεται να παραχθούν, ζήτημα με το οποίο ασχολούμαστε στην ενότητα 16.

4.4 Ασκήσεις

1. Ποια είναι η έξοδος του κώδικα που ακολουθεί;

```
int i = 0, j = 0;
System.out.print(i++ + " " + (++j) + " ");
System.out.println(i == j);
```

2. Ποια είναι η έξοδος του κώδικα που ακολουθεί; Εξηγήστε την απάντησή σας.

```
int k = 2;
System.out.println((k += 2) == (k *= 2));
```

3. Ποια είναι η έξοδος του κώδικα;

```
int i,j,k;
i = j = k = 0;
System.out.println((i++ == 0 || j++ == 0) && k++ == 0);
System.out.println(i + " " + j + " " + k);
```

4. Ποια είναι η έξοδος του κώδικα;

```
int i,j,k;
i = j = k = 0;
System.out.println(i++ == 0 || j++ == 0 && k++ == 0);
System.out.println("i++ == 0 || j++ == 0 && k++ == 0");
System.out.println(i + " " + j + " " + k);
```

5. Ποια είναι η έξοδος του κώδικα;

```
int i,j,k;
i = j = k = 0;
System.out.println(i++ == 0 && j++ == 0 || k++ == 0);
System.out.println("i++ == 0 && j++ == 0 || k++ == 0");
System.out.println(i + " " + j + " " + k);
```

6. Λάβετε από το πληκτρολόγιο 2 πραγματικούς αριθμούς και εμφανίστε στην οθόνη το γινόμενό τους. Εμφανίστε τα αποτελέσματα κατά τρόπο ώστε να προηγείται η περιγραφή τους και με 2 δεκαδικά ψηφία κατά μέγιστο.

Παράδειγμα

```
Δώσε αριθμό: 3.58
Δώσε αριθμό: 2.99
Το γινόμενο 3.58 x 2.99 = 10.7
```

7. Χρησιμοποιήστε μια μόνο println για να τυπώστε τη διεύθυνσή σας με την ακόλουθη μορφή:

```
Ονοματεπώνυμο
Οδός Αριθμός
Πόλη, ΤΚ
Χώρα
```

Υπόδειξη: Θυμηθείτε τους χαρακτήρες διαφυγής στην ενότητα 3.3.

8. Ο Γιώργος λαμβάνει μηνιαίο μισθό 1000 Ευρώ. Επίσης, λαμβάνει ενοίκιο από το διαμέρισμα στην Καλλιθέα 450 Ευρώ τον μήνα και το διπλάσιο ενοίκιο από το διαμέρισμα στου Παπάγου. Ο Γιώργος έχει ένα επιπλέον μηνιαίο εισόδημα. Τα έξοδα του μηνός Απριλίου έχουν ως εξής: Για τρόφιμα ξόδεψε τα 12/100 του συνολικού του εισοδήματος, τα καύσιμα κοστίζουν 0.2 Ευρώ το χιλιόμετρο, το Ηλεκτρικό ρεύμα 200 Ευρώ και το κόστος νερού 120 Ευρώ. Να αναπτυχθεί αλγόριθμος που ζητά από τον Γιώργο το επιπλέον μηνιαίο εισόδημα και τα χιλιόμετρα του Απριλίου και εμφανίζει κατάσταση εσόδων-εξόδων του Απριλίου σύμφωνα με το παρακάτω υπόδειγμα:

```
--------Κατάσταση Εσόδων-Εξόδων Απριλίου-----------------
Μισθός 1000
Ενοίκιο Καλλιθέας 450
Ενοίκιο Παπάγου 900
Επιπλέον μηνιαίο εισόδημα 100
-----------------------------------------------------
Συνολικό Εισόδημα 2450

Κόστος Τροφίμων 294.00
Κόστος Καυσίμων 160.00
Κόστος Ηλεκτρικού Ρεύματος 160.00
Κόστος Νερού 120
-----------------------------------------------------
Συνολικό Κόστος 774.00
*****************************************************
Διαφορά 1676.00
```

9. Ο Γιάννης είναι καπνιστής. Κάθε μέρα καταναλώνει ένα πακέτο τσιγάρα που κοστίζει 4 Ευρώ +20% ΦΠΑ. Επίσης, κάθε δύο μήνες χρειάζεται έναν αναπτήρα. Ρωτήστε τον χρήστη πόσο κοστίζει ένας αναπτήρας και υπολογίστε πόσο κοστίζει στον Γιάννη το κάπνισμα κάθε ημέρα και πόσο κάθε μήνα. Ποιο είναι το ετήσιο κόστος; Θεωρήστε πως όλοι οι μήνες αποτελούνται από 30 ημέρες. Για να υπολογίσετε το ημερήσιο κόστος επιμερίστε το κόστος του αναπτήρα στις ημέρες χρήσης του. Αν το ετήσιο εισόδημα του Γιάννη είναι 8.000 Ευρώ, πόσο τοις εκατό του εισοδήματός του ξοδεύει για κάπνισμα; Αν τα υπόλοιπα μηνιαία έξοδα του Γιάννη είναι 500 Ευρώ, τι ποσό από το εισόδημά του αποταμιεύει κάθε χρόνο;

Βιβλιογραφία

[1] “Operators (The Java™ Tutorials > Learning the Java Language > Language Basics).” https://docs.oracle.com/javase/tutorial/java/nutsandbolts/operators.html (accessed Sep. 14, 2021).

[2] “Formatting Numeric Print Output (The Java™ Tutorials > Learning the Java Language > Numbers and Strings).” https://docs.oracle.com/javase/tutorial/java/data/numberformat.html (accessed Sep. 14, 2021).

[3] K. Fogel, “If You’re Using Java’s Scanner Class for Keyboard Input, You’re Doing it Wrong - DZone Java,” DZone. https://dzone.com/articles/if-youre-using-javas-scanner-class-for-keyboard-in (accessed Sep. 14, 2021).

Κεφάλαιο 5

Σύνοψη
Στην ενότητα αυτή παρουσιάζονται οι δομές επιλογής και οι επαναληπτικές δομές της Java. Πιο συγκεκριμένα, παρουσιάζονται οι δομές επιλογής, if, if…else, switch και ο τριαδικός τελεστής. Επίσης, παρουσιάζονται οι επαναληπτικές δομές for, while, do-while και οι λέξεις-κλειδιά break και continue. Περιλαμβάνονται λυμένες ασκήσεις και ασκήσεις προς λύση.

Προαπαιτούμενη γνώση
Οι θεμελιώδεις τύποι και βασική γνώση του τύπου String. Βασική είσοδος και έξοδος. Η λειτουργία των τελεστών.

Λέξεις κλειδιά
Δομή επιλογής, επαναληπτική διαδικασία, βρόχος, τριαδικός τελεστής

5 Έλεγχος Ροής

Ο έλεγχος της ροής του προγράμματος πραγματοποιείται με τις δομές επιλογής και τις επαναληπτικές δομές. Οι δομές επιλογής μας δίνουν τη δυνατότητα να δημιουργούμε πολλαπλές διαδρομές μέσα στον κώδικά μας και να επιλέγουμε κάθε φορά την καταλληλότερη ανάλογα με τις συνθήκες που επικρατούν κατά την εκτέλεση του προγράμματος. Με τις επαναληπτικές δομές ρυθμίζουμε την επανάληψη τμημάτων του κώδικά μας, επίσης, ανάλογα με συγκεκριμένες συνθήκες που επικρατούν κατά την εκτέλεση του προγράμματος. Τόσο οι δομές επιλογής όσο και οι επαναληπτικές δομές αποτελούν βασικές δομές προγραμματισμού, όχι μόνο στην Java αλλά σε κάθε προγραμματιστικό περιβάλλον.
Επιπλέον, τόσο οι δομές επιλογής όσο και οι επαναληπτικές δομές βασίζονται στον έλεγχο μιας συνθήκης (condition). Η συνθήκη είναι μία λογική έκφραση, δηλαδή μία έκφραση τύπου boolean. Για παράδειγμα, η έκφραση i>5 && j<0 είναι λογική έκφραση, καθώς λαμβάνει την τιμή true ή false ανάλογα με την τιμή των i και j.

5.1 Δομές επιλογής

Οι δομές επιλογής μας δίνουν τη δυνατότητα να καθορίζουμε διαφορετικές διαδρομές που ακολουθεί ο κώδικάς μας ανάλογα με τις συνθήκες που επικρατούν στο πρόγραμμά μας κατά τον χρόνο εκτέλεσης. Πρόκειται για θεμελιώδεις δομές που συναντώνται με τη μία ή την άλλη μορφή σε κάθε γλώσσα προγραμματισμού. Για να αντιληφθεί κανείς τη σπουδαιότητά τους αρκεί να αναλογιστεί πως χωρίς αυτές τα προγράμματα θα ήταν μια συλλογή εντολών που θα εκτελούνταν αναγκαστικά σειριακά. Με άλλα λόγια, η λήψη αποφάσεων κατά την εκτέλεση του προγράμματος θα ήταν αδύνατη. Για παράδειγμα, αν ένα πρόγραμμα διέθετε ως δεδομένα μια σειρά από ονόματα και κάθε όνομα συνδεόταν με χαρακτηρισμό φύλου, δεν θα ήταν σε θέση να προσφωνήσει τους άντρες ως κυρίους και τις γυναίκες ως κυρίες.

Η Java υποστηρίζει τις δομές επιλογής if, if…else, switch και τον τριαδικό τελεστή. Είναι όμως εφικτό να συνδυαστούν οι δομές if και if…else διαμορφώνοντας έτσι μια ιδιαίτερη δομή γνωστή ως εμφωλευμένη (nested) if. Στη συνέχεια παρουσιάζονται μια προς μια οι δομές επιλογής της Java.

5.1.1 Η επιλογή if

Η γενική μορφή της if δίνεται ακολούθως:

```
if (condition) {
     statement(s);
}
```

Η δεσμευμένη λέξη if ακολουθείται από τη συνθήκη ελέγχου μέσα σε παρενθέσεις. Αν η συνθήκη είναι αληθής, η ροή του προγράμματος μπαίνει στο μπλοκ που ακολουθεί και εκτελεί τα statements. Αν η συνθήκη είναι ψευδής, η ροή του προγράμματος μεταπηδά στην έξοδο από το μπλοκ της if.

Μέσα στο μπλοκ της if μπορούμε να έχουμε ένα ή περισσότερα statements. Αν έχουμε μόνο ένα, οι αγκύλες είναι προαιρετικές, ωστόσο, συνηθίζεται να τοποθετούνται.

```
static void simpleIf() { //Κώδικας 5.1
    int i = 1, j = -1;
    if (i > 0 && j < 1) {
        System.out.println("i μεγαλύτερο από το 0");
        System.out.println("j μικρότερο από το 1");
    }
    System.out.println("Μετά τον έλεγχο της έκφρασης \"i>0 && j<1\"");
}
```

Κώδικας 5.1 *Παράδειγμα απλής if*

Στον κώδικα 5.1 η τιμή της συνθήκης ελέγχου εξαρτάται από τις τιμές των i και j. Αν για παράδειγμα, i==1 και j==-1, η συνθήκη είναι αληθής, οπότε οι προτάσεις "i μεγαλύτερο από το 0" και "j μικρότερο από το 1" θα εμφανιστούν. Αν όμως το i είναι μικρότερο ή ίσο με το 0 ή το j είναι μεγαλύτερο ή ίσο με το 1, η συνθήκη είναι ψευδής, οπότε αυτές οι προτάσεις δεν θα εμφανιστούν. Σε κάθε περίπτωση, θα τυπωθεί η πρόταση "Μετά τον έλεγχο της έκφρασης "i>0 && j<1"".

```
static void IfWithSimpleStatement() { //Κώδικας 5.2
    int i = 0;
    if (i > 0) {
        System.out.println("Το i είναι μεγαλύτερο από το 0");
    }
    System.out.println("Μετά τον έλεγχο της έκφρασης \"i>0\"");
}
```

Κώδικας 5.2 *Παράδειγμα απλής if με ένα statement*

Ο κώδικας 5.2 παρουσιάζει μια if που εφόσον η συνθήκη της αληθεύει εκτελεί μια μόνο πρόταση. Σε αυτήν την περίπτωση δεν είναι υποχρεωτικό να χρησιμοποιηθούν αγκύλες. Ωστόσο, στην πράξη, συχνά τοποθετούνται αγκύλες από πολλούς επαγγελματίες αλλά και από πολλά IDE, ακόμη και στην περίπτωση μίας πρότασης.
Η πρόταση που ακολουθεί μια if, μπορεί να είναι και η ίδια μια if. Έτσι προκύπτει μια δομή γνωστή ως εμφωλευμένη if (nested if).

```
static void nestedIf() { //Κώδικας 5.3
    int i = 1, j = -1;
    if (i > 0) {
        if (j < 1) {
            System.out.println("i>0 και j<1");
        }
    }
}
```

Κώδικας 5.3 *Παράδειγμα εμφωλευμένης if*

Ο κώδικας 5.3 ελέγχει καταρχάς αν το i είναι μεγαλύτερο του μηδενός. Αν η συνθήκη αληθεύει, προχωρά στην εκτέλεση της επόμενης πρότασης που είναι επίσης μια if. Η δεύτερη if ελέγχει αν το j είναι μικρότερο από το 1. Εφόσον η συνθήκη αυτή αληθεύει, τότε τυπώνει τη σειρά χαρακτήρων "i>0 και j<1".

5.1.2 Η επιλογή if…else

Η γενική μορφή της if…else έχει ως εξής:

```
if (condition) {
    statement(s);
}
else {
```

```
        statement(s);
    }
```

Στην περίπτωση που η συνθήκη αληθεύει εκτελούνται οι προτάσεις μέσα στο μπλοκ της if. Αν η συνθήκη είναι ψευδής, εκτελούνται οι προτάσεις μέσα στο μπλοκ της else. Τόσο οι προτάσεις μέσα στο μπλοκ της if όσο και οι προτάσεις στο μπλοκ της else μπορεί να αρχίζουν επίσης με μια if.

```
static void ifElseDemo() { //Κώδικας 5.4
    int i = 1, j = 2;
    if (i > 0) {
        if (j < 2 && j > 0) {
            System.out.println("j ίσο με 1");
        } else if (j < 2) {
            System.out.println("j<=0");
        }
        System.out.println("i>0");
    } else {
        System.out.println("i<=0");
    }
}
```

Κώδικας 5.4 *Παράδειγμα if…else*

Στον κώδικα 5.4 ελέγχεται καταρχάς η τιμή του i. Αν αυτή είναι θετική, η ροή συνεχίζει στο μπλοκ της εξωτερικής if, διαφορετικά η ροή διακλαδώνεται στον όρο else στο τέλος του κώδικα όπου τυπώνεται η σειρά "i<=0". Στην περίπτωση που η ροή εισέλθει στο μπλοκ της εξωτερικής if, ελέγχεται το j. Εφόσον η μεταβλητή j είναι ακέραιου τύπου και το j είναι μικρότερο του 2 και μεγαλύτερο του 0, δεν μπορεί παρά να είναι 1, οπότε τυπώνεται η σειρά "j ίσο με 1". Αν όμως η συνθήκη j < 2 && j>0 είναι ψευδής, δηλαδή το j δεν είναι 1, τότε ελέγχεται αν το j είναι μικρότερο του 2. Εφόσον το j είναι μικρότερο του 2 και δεν είναι 1, σημαίνει πως είναι μικρότερο ή ίσο του 0, οπότε τυπώνεται η σειρά "j<=0". Σε κάθε περίπτωση, μέσα στο μπλοκ της εξωτερικής if εκτυπώνεται η σειρά "i>0", ενώ στον όρο else της εξωτερικής if τυπώνεται η σειρά "i<=0".

5.1.3 Η switch

Σε μερικές περιπτώσεις ο έλεγχος της ροής του προγράμματος καθορίζεται από πολλαπλές συνθήκες. Όλες αυτές οι περιπτώσεις μπορούν να κωδικοποιηθούν με χρήση εμφωλευμένων if. Ωστόσο, ανάλογα με το πλήθος των συνθηκών, η χρήση εμφωλευμένων if αυξάνει την πολυπλοκότητα και καθιστά τον κώδικα δυσανάγνωστο. Σε κάποιες από αυτές τις περιπτώσεις ο κώδικας μπορεί να απλοποιηθεί με χρήση της switch.
Η γενική μορφή της switch έχει ως εξής:

```
switch(expression) {
    case constant1: statement(s); break;
    case constant2: statement(s); break;
    case constant3: statement(s); break;
    …
    …
    default: statement(s);
}
```

Η expression της switch πρέπει να είναι τύπου char, byte, short, int ή String. Οι τιμές ελέγχου στους όρους case πρέπει να είναι σταθερές, δηλαδή literals ή final μεταβλητές. Τα statement(s) όπως και στην if αντιπροσωπεύουν ένα ή περισσότερα statements. Ωστόσο εδώ στην περίπτωση που έχουμε περισσότερα από ένα statements δεν είναι υποχρεωτικό να τα εσωκλείσουμε σε αγκύλες. Η break ακολουθεί τα statements μίας case. Ωστόσο, η παρουσία της είναι προαιρετική ανάλογα με το αποτέλεσμα που θέλουμε να πετύχουμε. Πάντως, η break έχει ως αποτέλεσμα την έξοδο από την switch. Η default είναι επίσης προαιρετική.

Πώς δουλεύει όμως η switch; Καταρχάς αξιολογείται η expression. Ανάλογα με την τιμή της, ο έλεγχος της ροής κατευθύνεται στην κατάλληλη case. Αν σε καμία case δεν προβλέπεται η τιμή της expression, ο

έλεγχος της ροής κατευθύνεται στην default εφόσον υπάρχει αλλιώς έξω από την switch. Στην περίπτωση που ταιριάζει η τιμή της expression με την τιμή ελέγχου μιας case, εκτελούνται τόσο τα statements της αντιστοιχιζόμενης case όσο και τα statements των επόμενων case ή και της default έως ότου εκτελεστεί ένα break.

Η default δεν τοποθετείται υποχρεωτικά ως τελευταίος όρος της switch. Αντίθετα, μπορεί να τοποθετηθεί ως πρώτος όρος ή οπουδήποτε ανάμεσα στις case. Χρειάζεται όμως προσοχή ως προς το πού θα τοποθετήσουμε μια default πρόταση, γιατί σε συνδυασμό με τα break μπορεί να έχουμε εντελώς διαφορετικά αποτελέσματα. Αποσαφηνίζουμε παρακάτω με παραδείγματα χρήσης της switch:

```
static void switchDemo() { //Κώδικας 5.5
    int cV = 2;
    final int con = 3;
    switch (cV) {
        case 1:
            System.out.print("1");
            System.out.println("-case 1");
            break;
        case 2:
            System.out.println("2");
            break;
        case con:
            System.out.println("three");
            break;
        default:
            System.out.println("default");
    }
}
```

Κώδικας 5.5 *Τυπικό παράδειγμα χρήσης της switch*

Έστω στον κώδικα 5.5, η μεταβλητή cV είναι ακέραιου τύπου. Αν η τιμή της είναι 1, η έξοδος του κώδικα θα είναι 1-case 1, δηλαδή η τιμή της μεταβλητής αντιστοιχίζεται στην case 1 και επομένως εκτελούνται οι τρεις statements της case 1. Το πρώτο statement τυπώνει 1, το δεύτερο -case 1, ενώ το τρίτο statement είναι ένα break που τερματίζει την switch. Αν η cV έχει την τιμή 2, τότε τυπώνεται το 2. Αν η τιμή της cV είναι 3, τότε τυπώνεται η σειρά "three". Προσέξτε πως αν η μεταβλητή con δεν είχε δηλωθεί ως final, ο μεταγλωττιστής θα παρήγαγε κατάλληλο λάθος.
Τέλος, αν η τιμή της cV είναι διαφορετική και από το 1 και από το 2 και από το 3, τότε εκτελείται η default και εκτυπώνεται η σειρά “default”. Σημειώστε πως η default δεν ακολουθείται από break. Πράγματι, η τοποθέτηση ενός break στον τελευταίο όρο μιας switch δεν μεταβάλλει κατά κανένα τρόπο το αποτέλεσμα της switch, καθώς μετά τον τελευταίο όρο ακολουθεί έτσι κι αλλιώς έξοδος από την switch.

```
static void switchNoBreak() { //Κώδικας 5.6
    int cV = 1;
    final int con = 3;
    switch (cV) {
        case 1:
            System.out.print("1");
        case 2:
            System.out.print("2");
        case con:
            System.out.print("3");
        default:
            System.out.println("default");
    }

}
```

Κώδικας 5.6 *Παράδειγμα switch χωρίς break*

Αν η τιμή της cV είναι 1, η έξοδος του κώδικα 5.6 είναι 123default. Αν η τιμή της είναι 2, η έξοδος του κώδικα είναι 23default. Αν η τιμή της είναι 3, η έξοδος είναι 3default. Τέλος, αν η τιμή της cV είναι διάφορη και από το 1 και από το 2 και από το 3, η έξοδος είναι default. Επομένως, αν η break απουσιάζει, τότε η switch κατευθύνεται στην κατάλληλη case και από εκτελεί όλα τα statements έως ότου βρει break ή έως ότου τερματίσει η switch.
Όπως αναφέραμε προηγουμένως, η default δεν τοποθετείται υποχρεωτικά ως τελευταίος όρος μιας switch. Ωστόσο, η συμπεριφορά της switch δεν είναι ανεξάρτητη από τη θέση της default.

```
static void switchDefault() { //Κώδικας 5.7
    int cV = 1;
    final int con = 3;
    switch (cV) {
        default:
            System.out.print("default");
        case 1:
            System.out.print("1");
        case 2:
            System.out.print("2");
        case con:
            System.out.println("3");
    }
}
```

Κώδικας 5.7 *Παράδειγμα switch με την default ως πρώτο όρο*

Αν η τιμή της cV είναι 1, η έξοδος του κώδικα 5.7 είναι 123. Αν η τιμή της είναι 2, η έξοδος του κώδικα είναι 23. Αν η τιμή της είναι 3, η έξοδος είναι 3. Τέλος, αν η τιμή της cV είναι διάφορη και από το 1 και από το 2 και από το 3, η έξοδος είναι default123.
Ας δούμε και ένα παράδειγμα όπου η expression είναι τύπου String.

```
static void switchString() { //Κώδικας 5.8
    String monthLabel = "Jan";
    String monthAA = null;
    switch (monthLabel) {
        case "Jan":
            monthAA = "first";
            break;
        case "Feb":
            monthAA = "second";
            break;
        case "Mar":
            monthAA = "third";
            break;
        case "Apr":
            monthAA = "fourth";
            break;
        case "May":
            monthAA = "fifth";
            break;
        case "Jun":
            monthAA = "sixth";
            break;
        case "Jul":
            monthAA = "seventh";
            break;
        case "Aug":
            monthAA = "eighth";
            break;
        case "Sep":
            monthAA = "ninth";
```

```
                break;
            case "Oct":
                monthAA = "tenth";
                break;
            case "Nov":
                monthAA = "eleventh";
                break;
            case "Dec":
                monthAA = "twelfth";
        }
        if (monthAA == null) {
            System.out.println("Unkown Month Name: " + monthLabel);
        } else {
            System.out.println(monthLabel + " is the " + monthAA + " month of the
year");
        }
    }
```

Κώδικας 5.8 *Παράδειγμα switch με έκφραση ελέγχου τύπου String*

5.1.4 Ο τριαδικός τελεστής

Ο τριαδικός τελεστής που είδαμε ήδη στην ενότητα 4.1.5.3, αποτελεί μια συνοπτική δομή επιλογής. Παρουσιάζει μεγάλη ομοιότητα με τη δομή επιλογής if…else. Για παράδειγμα, ο κώδικας 5.9, επιτυγχάνει δύο φορές το ίδιο αποτέλεσμα, την πρώτη χρησιμοποιεί τον τριαδικό τελεστή και τη δεύτερη την δομή if…else.

```
    static void ternaryOp() { //Κώδικας 5.9
        int i = 1, j = 2, max;

        max = i > j ? i : j;

        if (i > j) {
            max = i;
        } else {
            max = j;
        }
    }
```

Κώδικας 5.9 *Παράδειγμα αναλογίας τριαδικού τελεστή και if…else*

5.2 Επαναληπτικές δομές

Οι επαναληπτικές δομές αποτελούν παρόμοιες με τις δομές επιλογής θεμελιώδεις δομές κάθε γλώσσας προγραμματισμού. Αναλογιστείτε πως χωρίς αυτές αν επιθυμούσαμε να τυπώσουμε τους ακέραιους από το 1 έως το 100.000, θα έπρεπε να πληκτρολογήσουμε 100.000 εντολές. Στην ουσία, ο αποτελεσματικός προγραμματισμός των Ηλεκτρονικών Υπολογιστών θα ήταν αδύνατος. Στα αγγλικά, η επαναληπτική δομή αναφέρεται με τον όρο loop που αποδίδεται στα ελληνικά ως βρόχος. Οι επαναληπτικές δομές που υποστηρίζει η Java είναι η while, η do-while και η for.

5.2.1 Η επαναληπτική δομή while

Η γενική μορφή της επαναληπτικής δομής while έχει ως εξής:

```
while(condition) {
    statement(s);
}
```

Η while εξετάζει τη συνθήκη condition και όσο αυτή είναι αληθής εκτελεί μια ή περισσότερες statements. Παρόμοια με την if, αν έχουμε μια μόνο statement, οι αγκύλες είναι προαιρετικές.

```
static void whileDemo() { //Κώδικας 5.10
    int i = 10;
    while (i > 0) {
        System.out.print("i>0 ");
        i--;
    }
    System.out.println("i=" + i);
}
```

Κώδικας 5.10 *Παράδειγμα απλής while*

Στον κώδικα 5.10 η ακέραιη μεταβλητή i αρχικοποιείται στην τιμή 10. Στη συνέχεια η while εξετάζει αν το i είναι μεγαλύτερο από το 0. Μέσα στο μπλοκ της while τυπώνεται το μήνυμα "i>0" και ακολούθως μειώνεται η τιμή του i κατά 1. Η διαδικασία επαναλαμβάνεται έως ότου το i πάψει να είναι μεγαλύτερο από το 0. Δεδομένου ότι το i αρχικοποιήθηκε στο 10 και η τιμή του μειώνεται κατά 1 σε κάθε επαναληπτικό βήμα, το i θα γίνει 0 ακριβώς μετά από 10 επαναλήψεις. Επομένως, ο κώδικας 5.10, τυπώνει 10 φορές τη σειρά "i>0", στη συνέχεια το i γίνεται 0, η συνθήκη i>0 καθίσταται ψευδής και ο κώδικας εξέρχεται από τον βρόχο while, οπότε εκτελεί την επόμενη πρόταση και τυπώνει i=0.

Προσέξτε πως αν στον κώδικα 5.10 η μεταβλητή i είχε αρχικοποιηθεί σε τιμή μικρότερη ή ίση με το 0, η συνθήκη ελέγχου θα ήταν εξαρχής ψευδής και ο κώδικας δεν θα εισερχόταν ποτέ στο μπλοκ της while. Αντίθετα, αν η μεταβλητή i δεν μειωνόταν μέσα στην while, τότε η ροή του κώδικα δεν θα εξερχόταν από τον βρόχο. Σε αυτήν την περίπτωση λέμε ότι έχουμε έναν ατέρμονο βρόχο (endless loop). Προφανώς, η παρουσία ατέρμονου βρόχου μέσα σε ένα πρόγραμμα αποτελεί σοβαρό προγραμματιστικό λάθος.

Τόσο η while όσο και οι υπόλοιπες επαναληπτικές δομές μπορούν να αξιοποιήσουν τις λέξεις-κλειδιά break και continue. Η μεν break μεταφέρει τη ροή του προγράμματος στην πρώτη πρόταση έξω από το μπλοκ της while, η δε continue μεταφέρει τη ροή στη συνθήκη ελέγχου.

Ας υποθέσουμε πως θέλουμε να τυπώσουμε 10 το πολύ τυχαίες ακέραιες τιμές στην κλίμακα 1 έως 10. Αν όμως κάποια από τις τιμές αυτές είναι μικρότερη από το 2, τότε να την τυπώσουμε και να σταματήσουμε ακόμη και αν δεν έχει συμπληρωθεί ο αριθμός των 10 εκτυπωμένων τιμών.
Ένας τρόπος για να πετύχουμε αυτήν την απαίτηση δίνεται στον κώδικα 5.11.

```
static void prtRandoms() {
    int i = 0, rI;
    Random generator = new Random();
    while (i < 10) {
        i++;
        rI = generator.nextInt(10) + 1;
        if (rI >= 2) {
            System.out.println(rI);
        } else {
            System.out.println(rI);
            break;
        }
    }
}
```

Κώδικας 5.11 *Παράδειγμα while με break*

Καταρχάς στον κώδικα 5.11 δηλώνουμε και αρχικοποιούμε τη μεταβλητή i στο 0. Παράλληλα δηλώνουμε τη μεταβλητή rI. Στη συνέχεια δημιουργούμε το αντικείμενο generator. Πρόκειται για μια γεννήτρια τυχαίων αριθμών. Σημειώστε πως για να χρησιμοποιήσετε ένα τέτοιο αντικείμενο θα πρέπει να εισάγετε στο πρόγραμμά σας τη βιβλιοθήκη Java.util.Random. Αυτό επιτυγχάνεται αν εισάγετε αμέσως μετά τη δήλωση του πακέτου τη σειρά import Java.util.Random; Στη συνέχεια αναπτύσσουμε μια επαναληπτική διαδικασία while. Η συνθήκη ελέγχου είναι i<10, δηλαδή η επαναληπτική διαδικασία θα επαναλαμβάνεται όσο το i θα είναι μικρότερο από το 10. Δεδομένου ότι το i αρχικοποιήθηκε στο 0 και σε κάθε επανάληψη αυξάνεται κατά 1, θα εκτελεστούν 10 το πολύ επαναλήψεις. Αυτό είναι συμβατό με την απαίτηση να τυπώσουμε 10 το πολύ τυχαίους ακέραιους. Στη

συνέχεια χρησιμοποιούμε τον generator για να παράξουμε έναν τυχαίο αριθμό. Η συνάρτηση nextInt παράγει έναν τυχαίο ακέραιο από το 0 έως την τιμή του ορίσματός της μείον 1, δηλαδή από το 0 έως το 9. Σε αυτόν τον αριθμό προσθέτουμε το 1, οπότε ο ακέραιος που εκχωρείται στη μεταβλητή rI είναι ένας τυχαίος από το 1 έως το 10. Μετά ελέγχουμε την τιμή της rI και αν είναι μεγαλύτερη ή ίση με το 2, απλώς την τυπώνουμε διαφορετικά, ισχύει η απαίτηση σύμφωνα με την οποία αν κάποια από τις τυχαίες τιμές είναι μικρότερη από το 2, τότε θα πρέπει να την τυπώσουμε και να σταματήσουμε περαιτέρω εκτύπωση τυχαίων αριθμών. Αυτό ακριβώς κάνει ο κώδικας 5.11. Στη συνέχεια τυπώνουμε την τιμή που είναι μικρότερη από το 2 και αμέσως μετά εκτελείται η break που διακόπτει την επαναληπτική διαδικασία και μεταφέρει τη ροή έξω από την επαναληπτική διαδικασία όπου ακολουθεί και η έξοδος της συνάρτησης prtRandoms.
Δύο σχόλια σχετικά με τον κώδικα 5.11. Γενικά είναι καλό στα προγράμματά μας να αποφεύγουμε τη φλυαρία, δηλαδή όπου είναι εφικτό να προτιμούμε τον βραχύτερο έναντι του μακρύτερου κώδικα. Προσέξτε πως ο κώδικας System.out.println(rI) εκτελείται και στις δύο διακλαδώσεις της if, δηλαδή είτε ισχύει η συνθήκη rI>=2 είτε όχι. Ένας τέτοιος κώδικας μπορεί να βγει πάντα έξω από την if έτσι ώστε μια παρουσία του να είναι αρκετή.

Ένα άλλο θέμα που θα μπορούσαμε να βελτιώσουμε σχετίζεται με την αύξηση του i. Αφού ελέγξουμε το i στην επόμενη γραμμή, το αυξάνουμε κατά 1. Το ίδιο ακριβώς αποτέλεσμα μπορούμε να πετύχουμε αν αυξήσουμε το i στη συνθήκη ελέγχου με χρήση του μεταθεματικού τελεστή προσαύξησης. Προσοχή όμως! Αν η αύξηση γίνει όπως γίνεται στον κώδικα 5.11, δεν έχει ιδιαίτερη σημασία αν χρησιμοποιηθεί ο προθεματικός ή ο μεταθεματικός τελεστής προσαύξησης. Αν όμως η αύξηση γίνει μέσα στη συνθήκη, τότε η χρήση του μεταθεματικού τελεστή διαφοροποιεί ουσιαστικά τη συμπεριφορά του κώδικα. Μελετήστε πώς την αλλάζει και γιατί. Ο κώδικας 5.12 παράγει το ίδιο ακριβώς αποτέλεσμα με τον κώδικα 5.11 με το πλεονέκτημα ότι είναι βραχύτερος.

```
static void prtRandoms() { //Κώδικας 5.12
    int i = 0, rI;
    Random generator = new Random();
    while (i++ < 10) {
        rI = generator.nextInt(10) + 1;
        System.out.println(rI);
        if (rI < 2) {
            break;
        }
    }
}
```

Κώδικας 5.12 *Βραχύτερη έκδοση της συνάρτησης prtRandoms*

Ας υποθέσουμε τώρα πως θέλουμε να τυπώσουμε 10 τυχαίες ακέραιες τιμές στην κλίμακα 1 έως 10 εξαιρώντας τις τιμές 3 και 4, δηλαδή 10 τιμές που ανήκουν στο σύνολο {1..10}-{3,4}. Η λέξη-κλειδί continue θα μας βοηθήσει στην επίλυση αυτού του προβλήματος.

```
static void prt10Randoms() { //Κώδικας 5.13
    int i = 0, rI;
    Random generator = new Random();
    while (i < 10) {
        rI = generator.nextInt(10) + 1;
        if (rI == 3 || rI == 4) {
            continue;
        }
        System.out.println(rI);
        i++;
    }
}
```

Κώδικας 5.13 *Παράδειγμα while με χρήση της λέξης-κλειδί continue*

Μέσα στον βρόχο while του κώδικα 5.13 παράγουμε καταρχάς έναν τυχαίο αριθμό από 1 έως 10. Στη συνέχεια ελέγχουμε αν πρόκειται για αριθμό που εξαιρείται. Αν όντως πρόκειται για αριθμό που εξαιρείται, καλούμε την continue ώστε να μεταφέρουμε τη ροή στην αρχή του βρόχου όπου ελέγχεται η συνθήκη i<10. Αν δεν πρόκειται για αριθμό που εξαιρείται, προχωρούμε κανονικά στην εκτύπωσή του και στη συνέχεια αυξάνουμε τη

μεταβλητή ελέγχου i κατά 1. Προσοχή, εδώ η i θα ήταν λάθος να αυξηθεί μέσα στη συνθήκη ελέγχου. Κάτι τέτοιο θα είχε ως αποτέλεσμα την παραγωγή συνολικά 10 ακέραιων και την εκτύπωση μόνο όσον είναι διάφοροι από το 3 και το 4, δηλαδή να εκτυπωθούν πιθανότατα λιγότεροι από 10 ακέραιους που προσδιορίζουν οι απαιτήσεις του προβλήματός μας.

5.2.2 Η επαναληπτική δομή do-while

Η γενική μορφή της do-while έχει ως εξής:

```
do {
     statement(s);
} while (condition);
```

Η διαφορά της από την while είναι πως η συνθήκη ελέγχεται στο τέλος. Επομένως, η do-while εγγυάται πως ο κώδικας μέσα στο σώμα της θα εκτελεστεί τουλάχιστον μία φορά.

Ας υποθέσουμε πως θέλουμε να λάβουμε μια σειρά από ακέραιους από τον χρήστη και να υπολογίσουμε το άθροισμά τους και επιπλέον, να σταματήσουμε τη διαδικασία εισόδου από τον χρήστη όταν αυτός δώσει την τιμή 0.

Σε αυτό το πρόβλημα, ο βρόχος do-while θα μας φανεί πιο χρήσιμος από τον βρόχο while, καθώς είναι αναγκαίο να λάβουμε μια τουλάχιστον είσοδο από τον χρήστη.

```
static void calcSum() { //Κώδικας 5.14
    int i;
    int sum = 0;
    Scanner in = new Scanner(System.in);
    do {
        System.out.print("Enter int (0 terminates input):");
        i = in.nextInt();
        sum += i;
    } while (i != 0);
    System.out.println("Sum = " + sum);
}
```

Κώδικας 5.14 *Παράδειγμα χρήσης του βρόχου do-while*

Στον κώδικα 5.14 ορίζουμε τη μεταβλητή i που θα χρησιμοποιήσουμε για να καταχωρούμε την είσοδο του χρήστη. Τη μεταβλητή i δεν είναι αναγκαίο να την αρχικοποιήσουμε, καθώς είναι βέβαιο πως πριν χρησιμοποιηθεί θα εκχωρηθεί τιμή σε αυτήν. Στη συνέχεια ορίζουμε τη μεταβλητή sum. Είναι αναγκαίο να αρχικοποιήσουμε την sum στο 0, καθώς αυτή προσαυξάνεται κατά την είσοδο του χρήστη σε κάθε βήμα της επαναληπτικής διαδικασίας do-while. Ακολούθως ορίζουμε ένα αντικείμενο της κλάσης Scanner για να το χρησιμοποιήσουμε για είσοδο από τον χρήστη. Στο επόμενο βήμα μπαίνουμε στην επαναληπτική διαδικασία do-while όπου ζητάμε και παίρνουμε έναν ακέραιο από τον χρήστη με τον οποίο ενημερώνουμε τον αθροιστή sum. Στο τέλος της do-while γίνεται ο έλεγχος της συνθήκης. Αν ο χρήστης έδωσε ακέραιο διάφορο του 0, η διαδικασία επαναλαμβάνεται αλλιώς τερματίζεται.

Όλες οι επαναληπτικές διαδικασίες μπορούν να υλοποιηθούν με χρήση οποιασδήποτε επαναληπτικής δομής. Ωστόσο, κάποιες δομές παρέχουν διευκολύνσεις ανάλογα με τις απαιτήσεις που πρέπει να υλοποιηθούν. Στον κώδικα 5.15 επιλύουμε το ίδιο πρόβλημα με αυτό του κώδικα 5.14 αλλά χρησιμοποιώντας τη δομή while αντί της δομής do-while.

```
static void calcSum() { //Κώδικας 5.15
    int i;
    int sum = 0;
    Scanner in = new Scanner(System.in);
    System.out.print("Enter int (0 terminates input):");
    i = in.nextInt();
    while (i != 0) {
        sum += i;
        System.out.print("Enter int (0 terminates input):");
```

```
            i = in.nextInt();
        }
        System.out.println("Sum = " + sum);
    }
```

Κώδικας 5.15 *Υλοποίηση της calcSum με while αντί do-while*

Προσέξτε πως στον κώδικα 5.15 οι εντολές εισόδου από τον χρήστη εμφανίζονται δύο φορές, μία έξω από τον βρόχο και μια μέσα στον βρόχο μετά την επεξεργασία που στην περίπτωσή μας είναι η πρόσθεση του ακέραιου i στον αθροιστή sum. Πρόκειται για μια τυπική μεταφορά ενός do-while σε while. Ωστόσο, η διπλή παρουσία των εντολών εισόδου μπορεί να αποφευχθεί αν οργανώσουμε τον κώδικά μας όπως φαίνεται στον κώδικα 5.16.

```
    static void calcSum() { //Κώδικας 5.16
        int i;
        int sum = 0;
        Scanner in = new Scanner(System.in);
        while (true) {
            System.out.print("Enter int (0 terminates input):");
            i = in.nextInt();
            if (i == 0) {
                break;
            }
            sum += i;
        }
        System.out.println("Sum = " + sum);
    }
```

Κώδικας 5.16 *Προσομοίωση της do-while με while*

Στον κώδικα 5.16 προσομοιώνουμε τη λειτουργία της do-while με χρήση της while. Η ροή του κώδικα θα εισέλθει οπωσδήποτε μια τουλάχιστον φορά στον βρόχο while, καθώς η συνθήκη ελέγχου είναι πάντα true. Στην ουσία ο έλεγχος του βρόχου γίνεται από την if, η οποία αν εντοπίσει τιμή ίση με το 0, καλεί την break που προκαλεί έξοδο από τον βρόχο.
Συχνά δημιουργείται η αντίληψη ότι η continue κατευθύνει τη ροή του προγράμματος στην αρχή της επαναληπτικής διαδικασίας. Η αντίληψη αυτή όμως είναι λανθασμένη. Η continue κατευθύνει τη ροή στη συνθήκη ελέγχου του βρόχου. Αν ο βρόχος είναι τύπου while, τότε η αρχή της διαδικασίας συμπίπτει με τη συνθήκη ελέγχου. Αν όμως ο βρόχος είναι τύπου do-while, τότε η συνθήκη βρίσκεται στο τέλος. Ο κώδικας 5.17 επιδεικνύει τη λειτουργία της continue σε έναν do-while βρόχο.

```
    static void checkContinue() {
        int i = 0;
        do {
            i++;
            System.out.println(i);
            if (i == 1) {
                continue;
            }
            System.out.println(i);
        } while (i < 1);
    }
```

Κώδικας 5.17 *Η λειτουργία της continue*

Ο κώδικας 5.17 όταν εκτελείται τυπώνει 1. Το αποτέλεσμα του κώδικα είναι απόδειξη ότι η continue μεταφέρει τη ροή στη συνθήκη ελέγχου και όχι απλώς στην αρχή του βρόχου. Με την είσοδο μέσα στον βρόχο, η μεταβλητή i αυξάνεται κατά 1 και γίνεται 1 από 0 που ήταν η τιμή στην οποία αρχικοποιήθηκε στην πρώτη γραμμή της checkContinue. Στη συνέχεια τυπώνεται η τιμή της i, δηλαδή το 1. Ακολούθως ελέγχεται η τιμή της i, η συνθήκη i==1 βρίσκεται αληθής και εκτελείται το continue. Αν το continue κατηύθυνε τη ροή στην αρχή του βρόχου, τότε το i θα γινόταν 2 και στη συνέχεια θα τυπωνόταν πριν φτάσει στη συνθήκη ελέγχου και

εξέλθει από τον βρόχο. Όμως, η continue κατευθύνει άμεσα τη ροή στη συνθήκη ελέγχου η οποία φυσικά για την τιμή του i ίση με το 1 είναι ψευδής και έτσι ο βρόχος διακόπτεται έχοντας εκτυπώσει μόνο το 1.

5.2.3 Η επαναληπτική δομή for

Η for παρουσιάζει μεγάλη ευελιξία και είναι ίσως η συχνότερα χρησιμοποιούμενη επαναληπτική δομή. Η γενική μορφή της έχει ως εξής:

```
for (initialization; condition; statement) {
     statement(s);
}
```

Η for επομένως αποτελείται από τέσσερα μέρη. Το initialization είναι το τμήμα εκείνο στο οποίο τυπικά αρχικοποιείται ή και δηλώνεται η μεταβλητή ελέγχου του βρόχου. Η condition είναι η συνθήκη που ελέγχει αν θα συνεχιστεί η επαναληπτική διαδικασία ή όχι. Το statement μπορεί να είναι μια οποιαδήποτε εντολή ή συνάρτηση. Στην τυπική περίπτωση, στο statement μεταβάλλεται η τιμή της μεταβλητής ελέγχου. Το τέταρτο μέρος είναι το μπλοκ της for στο οποίο μπορούμε να τοποθετήσουμε ένα ή περισσότερα statements.

Το initialization εκτελείται μια φορά κατά την είσοδο στον βρόχο. Στη συνέχεια εκτελείται η condition. Αν βρεθεί αληθής, η ροή κατευθύνεται στο μπλοκ της for. Αφού εκτελεστούν οι statements του μπλοκ, η ροή κατευθύνεται στο statement εντός του κύριου σώματος της for. Μετά την εκτέλεση του statement εντός του κύριου σώματος, η ροή κατευθύνεται στην condition για να ακολουθήσει μια κυκλική διαδρομή του τύπου condition-statement(s)-statement έως ότου βέβαια η condition λάβει την τιμή false. Σε αυτήν την περίπτωση, διακόπτεται η for και η ροή του προγράμματος κατευθύνεται στην έξοδο του βρόχου.
Ας δούμε καταρχάς μια απλή εφαρμογή της for. Ο κώδικας 5.18, τυπώνει τους ακεραίους από 0 έως 9.

```
for (int i = 0; i < 10; i++) {
     System.out.println(i);
}
```

Κώδικας 5.18 *Εκτύπωση των ακέραιων 0..9 με την for*

Η πρώτη τιμή που εκτυπώνεται είναι η τιμή 0. Αυτό δείχνει πως η ροή εκτέλεσης πρώτα εισέρχεται στο μπλοκ της for και στη συνέχεια εκτελεί την προσαύξηση του i στο κυρίως σώμα της for. Η δε τελευταία τιμή είναι το 9. Πράγματι, μόλις τυπωθεί το 9, η ροή κατευθύνεται στην πρόταση προσαύξησης του i που γίνεται 10. Στη συνέχεια, η συνθήκη ελέγχου καθίσταται ψευδής και οδηγεί στην έξοδο του βρόχου.
Ας σημειωθεί πως στο τμήμα αρχικοποίησης της for δεν είναι υποχρεωτικό να δηλώνεται η μεταβλητή ελέγχου. Ωστόσο, αποτελεί καλή πρακτική η δήλωσή της εφόσον αυτή δεν είναι χρήσιμη έξω από την for. Κάτι ακόμη που πρέπει να αποσαφηνίσουμε είναι πως κανένα από τα τέσσερα τμήματα της for δεν είναι υποχρεωτικό.

Στη συνέχεια δίνουμε παραδείγματα χρήσης της for μέσα από τα οποία γίνονται σαφείς οι παρατηρήσεις μας σε σχέση με αυτήν. Πιο συγκεκριμένα, υλοποιούμε με for κώδικες που αξιοποιούν την while ή την do-while και έχουν ήδη παρουσιαστεί μέχρι εδώ σε αυτήν την ενότητα. Με αυτόν τον τρόπο θα αποκτήσουμε πληρέστερη εικόνα της σχετικής ευελιξίας της for.
Ο κώδικας 5.10 που επιδεικνύει τη χρήση της while, υλοποιείται με for όπως δείχνει ο κώδικας 5.19.

```
static void forDemo() { //Κώδικας 5.19
    int i;
    for (i = 10; i > 0; i--) {
        System.out.print("i>0 ");
    }
    System.out.println("i=" + i);
}
```

Κώδικας 5.19 *Ίδιο αποτέλεσμα με την while του κώδικα 5.10*

Στον κώδικα 5.19 η μεταβλητή ελέγχου i δεν δηλώνεται στο τμήμα αρχικοποίησης της for. Αυτό είναι αναγκαίο, καθώς θέλουμε να προσπελάσουμε τη μεταβλητή έξω από τα όρια της for. Αν την είχαμε δηλώσει στο τμήμα αρχικοποίησης της for, η εμβέλεια της μεταβλητής θα περιοριζόταν στα τμήματα της for.

Η συνάρτηση prtRandoms που παρουσιάζεται στον κώδικα 5.12 μεταγράφεται με for όπως φαίνεται στον κώδικα 5.20:

```
static void prtRandoms() { //Κώδικας 5.20
    Random generator = new Random();
    int rI;
    for (int i = 0; i++ < 10;) {
        rI = generator.nextInt(10) + 1;
        System.out.println(rI);
        if (rI < 2) {
            break;
        }
    }
}
```

Κώδικας 5.20 *Η prtRandoms του κώδικα 5.12 υλοποιημένη με for*

Προσέξτε πως σε αυτήν την περίπτωση το τρίτο τμήμα της for δεν χρησιμοποιήθηκε. Όπως αναφέραμε προηγουμένως, όλα τα τμήματα της for είναι προαιρετικά.
Η συνάρτηση prtRandoms που παρουσιάστηκε στον κώδικα 5.13 μεταγράφεται με for όπως δείχνει ο κώδικας 5.21:

```
static void prtRandoms() { //Κώδικας 5.21
    int rI;
    Random generator = new Random();
    for (int i = 0; i < 10; i++) {
        rI = generator.nextInt(10) + 1;
        if (rI == 3 || rI == 4) {
            continue;
        }
        System.out.println(rI);
    }
}
```

Κώδικας 5.21 *Η prtRandoms του κώδικα 5.13 υλοποιημένη με for*

Ο κώδικας 5.22 υλοποιεί την calcSum του κώδικα 5.14 χρησιμοποιώντας for αντί για do-while:

```
static void calcSum() { //Κωδικας 5.22
    int sum = 0;
    Scanner input = new Scanner(System.in);
    for (int i = 1; i != 0;) {
        System.out.print("Enter int (0 terminates input):");
        i = input.nextInt();
        sum += i;
    }
    System.out.println("Sum = " + sum);
}
```

Κώδικας 5.22 *Η calcSum του κώδικα 5.14 υλοποιημένη με for*

Ο κώδικας 5.23 παρέχει μια κάπως διαφορετική υλοποίηση του κώδικα 5.18:

```
 for (int i=0; i<10; System.out.println(i));
```

Κώδικας 5.23 *Εναλλακτική υλοποίηση του κώδικα 5.18*

Συχνά οι αρχάριοι προγραμματιστές έχουν την εντύπωση πως οι μεταβλητές ελέγχου της for είναι υποχρεωτικά τύπου int. Κάτι τέτοιο βέβαια δεν αληθεύει. Παρακάτω στον κώδικα 5.24, εκτυπώνουμε το αγγλικό αλφάβητο με κεφαλαίους χαρακτήρες:

```
static void prtAlpha() { //Κώδικας 5.24
```